DP2. Certified caches on time laws, and drivers that move frames

Cache validity windows from bounded closed-form motion, two event queues, and moving frames that reach rays through a grid of their own

Defensive publication · draft 2 · 23 October 2026 · Author: Kyle Kimberlin (AI assistants helped write it) · Licence: CC BY 4.0

Keywords: cache invalidation; validity interval; certificate; kinetic data structure; event queue; deterministic simulation; closed-form motion; procedural animation; envelope; Lipschitz bound; threshold crossing; fixed-point arithmetic; bit-exact CPU and GPU evaluation; scene graph; transform hierarchy; nested frames; object-space instances; pose at display time; modal vegetation sway; swept bounding box; motion bound over a time horizon; two-level acceleration structure; ray tracing moving objects; shadow and lighting caches; simulation level of detail; virtual reality.

Classification (CPC): G06T 13/20 (three-dimensional animation); G06T 13/60 (3D animation of natural phenomena, e.g. rain, snow, water or plants); G06T 15/06 (ray-tracing); G06T 15/60 (shadow generation); G06T 17/005 (tree description, e.g. octree, quadtree); G06T 19/00 (manipulating 3D models or images for computer graphics); G06T 2210/12 (bounding box); G06T 2210/61 (scene description); A63F 13/52 (game output involving aspects of the displayed game scene); A63F 13/57 (simulating properties, behaviour or motion of objects in the game world); A63F 2300/64 (computing dynamical parameters of game objects, e.g. motion determination); A63F 2300/66 (rendering three-dimensional images).

Related disclosures in this series: DP1 (sun visibility decided from the light's side); DP3 (exact integer time, whole-number phases, the law network mode); DP5 (the physel record and drawing without meshes).

1. Field

Real-time rendering and simulation of three-dimensional worlds in which much never stops moving (the sun, wind, water, plants, people): keeping cached results valid while things move, and storing moving matter so that motion costs little, for games, virtual reality and any interactive or offline simulation.

2. Problem and known methods

Most caches assume a still world. When something moves, an engine rebuilds the cache, or keeps it and accepts that it is wrong for a while. Without a method like the one below, one swaying tree can switch off sun-shadow caches everywhere; every secondary ray tests every moving group of records; every moving record is decoded again each frame; and each change of the sun rebuilds the sky and shadow bounds and wakes every light probe, so a sun at its real rate costs what a sun many times faster does.

Known method	What it lacks here
Dirty flags and scene-graph caches [10, 11, 12]: a change marks data stale; it is recomputed when read	No foresight: constant motion makes everything stale every frame
Shadow-cache invalidation by mover class [13]: pages dropped when a primitive moves; authors may suppress it	A choice between cost and stale shadows; no proof of how long a page stays true
Procedural motion in shaders [14, 15, 16]: sway evaluated per vertex from float time	Every cache downstream must assume motion anywhere; CPU and GPU disagree; float time drifts
Validity intervals in animation tools [9]: each evaluated object reports how long its cached result holds	Each plug-in states its interval by hand; not derived from bounds of the motion; not used in ray-tracing structures or event queues
Kinetic data structures [1, 2, 23, 24]: facts hold until	Motion plans solved for roots; no bit-exact CPU and

Known method	What it lacks here
certificates fail; failure times wait in a priority queue	GPU evaluators; one queue for everything
Conservative advancement and bounds over time [18, 21, 22]: a speed bound gives a safe time before contact	Used for contact only
Space-time ray tracing and temporal coherence [3, 4]: objects bounded in four dimensions	Offline; a whole shot's motion known in advance
Two-level ray-tracing structures [5, 6, 7, 8]: objects keep their own structures; a top level over instances is rebuilt or refit each frame	The top level never knows how long a listing stays true
Simulation culling [17]: unseen objects are not simulated; their state on return comes from statistics	Driven by visibility; sampled, not exact
Discrete-event and optimistic simulation [19, 20]: events in time order, rollback when one arrives late	Orders processes, not caches

None gives all four of: (a) every changing quantity as a small closed-form law with a proven envelope and largest rate, evaluated with the same bits on any CPU and GPU; (b) from those bounds, for each cached fact, the earliest step at which it could stop being true; (c) two queues, one deciding results identically on every machine, one only scheduling cache work; (d) matter that never moves inside its frame, with moving frames entering rays through a grid rebuilt only when a certificate runs out.

3. Summary

Every changing value, frame and cache has a time form: Still, Law (a closed formula of time), Live (stepped by a solver) or Folded (nobody there). A law is a few 32-bit words; integer-only evaluators give the same bits on CPU and GPU, and each law carries an envelope and a largest rate worked out from the evaluator's own arithmetic. Each cached fact carries a certificate, the earliest step at which a law it read could leave the band that keeps it true, found by inverting the law or by stepping by its largest rate; certificates wait in a per-machine cache queue, apart from a per-area queue of events that decide results in a fixed order. Matter rests in frames that drivers move: a record never changes while its object moves, a pose pass places every law-driven frame once a frame at the predicted display time, and a grid of frames listed by their swept boxes over a horizon brings moving frames into rays. Caches carry validity windows and are never saved. Caches of yes-or-no facts change the cost, never the picture; continuous caches stay within a written budget of one eighth of an output step of a fresh computation.

4. Detailed description

4.1 Words used

Word	Meaning
Step; tick	1/480 s; 1/2,048 of a step
Law	A closed formula of time stored as 32-bit words; its value v means v × 2^e
Envelope; largest rate	The lowest and highest value a law takes; the most it changes between two times at most a step apart
Band; certificate; window	The values that keep a cached fact true; the earliest step it could stop being true; the steps it is proven true
Frame; driver; holder	A coordinate system that can move; what moves frames; the world, or the frame no law moves, at a chain's root
Horizon; sweep	A span of steps; a box holding a frame's box wherever its laws take it over a horizon
Physel; chunk	A 32-byte record of a small solid piece of matter (DP5); up to 64 physels of one object, for culling
x ≫ k; x ≪ k	x shifted right (rounded down) or left by k bits

Time (DP3): a signed 64-bit count of steps since a fixed UTC epoch plus an 11-bit fraction; the 32-bit time word $t32 = (\text{step} \times 2^{11} + \text{fraction}) \bmod 2^{32}$; a wave of n whole cycles per 2^{21} steps has the phase $n \times t32 + \varphi$ in 2^{-32} turns.

4.2 The four time forms

Every value, driver and cache has a time form (Figure 1): drivers carry it, and blocks of changing state may carry it as a 2-bit tag.

Form	Meaning	Cost a frame
Still	Constant	0
Law	f(parameters, t − t0), a closed form: seven kinds and a modal instance (§4.3, §4.15)	Only where something reads it
Live	Stepped by a solver on a power-of-two ladder of the step (1/480 to 1/30 s), per island of drivers, never per physel	The solver's
Folded	The area is empty; laws run free; events wait and replay in order on return (§4.8)	0

Rules: forms switch only for physical causes (contact, impulse, strain, energy, temperature, intent), never for the camera; a switch is an event in the area's queue, with hysteresis; Law → Live starts from the law's state at that step; Live → Law is exact for linear motion (a forced response plus a closed-form free decay). Live solvers publish short law segments (a polynomial until the next contact), so drawing, sound and the network read one interface. A law nobody reads is never evaluated. Only drawing detail may follow the view; physics, caches and the network use the law itself.

4.3 Laws as fixed words

An area keeps one **law table**: an array of 32-bit words, shared unchanged by the CPU and the GPU. A law is named by the index of its first word. Header (Figure 2):

Word	Bits	Holds
0	0-2	Kind (0-6 below; 7, the modal instance, §4.15)
0	3	Clock: 0 physical seconds (cooling, sway); 1 world time (sun, weather; a world may run at a rational rate)
0	4-9	Exponent e, signed, −32 to 31
0	10-20; 21-31	The 11-bit fractions of t0 and of the end
1-2; 3-4		The steps of t0 and of the end (i64, low word first)
5; 6		Envelope: lowest; highest (i32)
7		Largest rate, value units a step (u32)
8 on		The kind's parameters

Outside [t0, end] a law reads its clock as t0 or the end, so it holds its first and last values.

Kind	Parameter words	Value
0 Constant	c	c
1 Polynomial	s | f1 ≪ 8 | f2 ≪ 16 | f3 ≪ 24; c0; c1; c2; c3	τ = min((ticks since t0) ≫ s, $2^{31} − 1$); u = c3, then u = sat(c2 + (uτ ≫ f3)), sat(c1 + (uτ ≫ f2)), sat(c0 + (uτ ≫ f1)); 64-bit products; sat clamps to i32
2 Spectral sum	n; c; per wave a, n_i, φ_i	sat(c + ((Σ a_i · sin(n_i · t32 + φ_i)) ≫ 30)), summed in 64 bits in word order
3 Damped modes	m | s ≪ 16 | r ≪ 24; c; per mode a, n_i, φ_i, λ_i	The spectral sum, each a decayed from t0
4 Relaxation, banded	n; per band: start (ticks after t0, 2 words), from, toward, λ, s | r ≪ 8	In the last band started: toward + decay(from − toward)
5 Periodic table	k | m ≪ 8; 2^m samples	The samples over 2^k steps from t0, repeating, interpolated on 20 bits
6 Composite	n; per term: law a, law b or none, shift	sat(Σ (a · b) shifted), summed in 64 bits; factors are never composites

Whole-number frequencies make waves loop exactly, as in Tessendorf's ocean [26]. **Decay:** x = (λ × τ) ≫ r halvings in Q32.32, with τ = min((ticks since t0) ≫ s, $2^{32} − 1$); 2^−x = m × 2^−(30+k), k the whole part of x and m from a table; the result is ⌊|a| × m / 2^(30+k)⌋ with a's sign, rounded toward zero so

either sign ends at exactly 0; 64 or more halvings give 0 by a test, never by a shift of 64. A maker picks s from the longest half-life and r from the shortest.

Limits: at most 64 waves, modes, bands or terms; periodic tables $m \le 16$, $k \le 53$, $m \le k + 11$. A four-mode damped law is 26 words. Further kinds may be added: **path** (splines, timetables, gait phase) and **ephemeris** (orbits of the sun, moon and planets). A law is made only by a maker that checks it and works out its envelope and largest rate (§4.5); a maker refuses a law whose values could leave i32 (or a sum i64) anywhere in its window, so nothing is ever clamped inside a window and the stated bounds are true.

4.4 Evaluators with a CPU twin and a GPU twin

One evaluator per kind, written for the CPU and as a compute-shader function, both reading only the words. The twins' rules: integers only; on the GPU 32-bit types only, 64-bit values as word pairs with explicit carry and 64-bit products from 32 × 32 extended multiplies (no optional 64-bit shader feature); no division on the GPU (results differ by vendor); no shift by the word's width or more (such cases written out); no recursion (composites are flat); wrapping arithmetic only where a wrap is meant; sums in word order; maths tables made once by integer-only code and uploaded as a buffer, never compiled into shaders (a shader compiler may copy such a table into every thread's memory). Parity means equal bits: a difference is a bug found at its cause, never a tolerance.

Fixed-point functions: the sine of a phase in 2^{-32} turns, in Q1.30, from a 4,097-entry table on the phase's top 12 bits, interpolated on the next 20 (error below 3.2×10^{-7}: $h^2/8$ for linear interpolation with $h = 2\pi \div 4{,}096$, plus rounding); the cosine as the sine a quarter turn on; 2^−x from a 1,025-entry table on the top 10 bits of x's fraction, interpolated on the next 20 (relative error below 2×10^{-7}). The tables take 24,588 bytes.

```
value(T, at, clocks):                              // T: the law table's words
  if kind(T[at]) = COMPOSITE:
      sum ← 0                                       // 64-bit, wrapping, in word order
      for each term (a, b, shift):
          p ← base(T, a) × (b ≠ NONE ? base(T, b) : 1)
          sum ← sum + (shift ≥ 0 ? p ≫ shift : p ≪ −shift)
      return sat32(sum)
  return base(T, at)
base(T, at):                                        // the modal instance: §4.15
  t ← clamp(clocks[clock bit], t0, end); d ← ticks from t0 to t   // held at 2⁶⁴ − 1
  CONSTANT c | POLYNOMIAL Horner | SPECTRAL sat32(c + ((Σ aᵢ·sin(nᵢ·t32 + φᵢ)) ≫
30))
  DAMPED: the same with aᵢ ← decay(aᵢ, λᵢ, d)
  RELAXATION: j ← last band with startⱼ ≤ d; towardⱼ + decay(fromⱼ − towardⱼ, λⱼ, d −
startⱼ)
  PERIODIC: pos ← d mod 2^(k+11); i ← pos ≫ (k + 11 − m); f ← the next 20 bits
            sample[i] + (sample[(i+1) mod 2^m] − sample[i]) · f / 2²⁰, rounded toward
sample[i]
```

4.5 Envelopes and largest rates

Each maker works out both bounds from the evaluator's own integer operations, rounding and table error included, so they hold for every input; they may be loose. Over k steps a law changes by at most k times its rate. Every rate is at most the envelope's width and at most $2^{32} - 1$.

Kind	Envelope	Largest rate
Polynomial	Horner over intervals, τ in [0, T]	A step moves τ by Δ = max(2,048 ≫ s, 1); each stage changes by at most ((max|u| · Δ + its input's change · T) ≫ f) + 1
Spectral, damped	c ± Σ|a| (refused above 2^{31})	1 + ((Σ (|a| · W(n) + D · 2^{30})) ≫ 30), with D = 0 for a spectral sum
Relaxation	Between from and toward of each band starting in the window	The largest of each band's D (below, with a = from − toward and the band's λ, s and r) and, across each switch, both bands' D plus the jump

Kind	Envelope	Largest rate
Periodic table	Lowest and highest sample	Largest neighbour difference × samples a step spans (at least 2^{-20} of one) + 2
Composite	Products of the factors' envelopes (four corners), shifted, summed	$\lvert\Delta a\rvert \cdot \max\lvert b\rvert + \max\lvert a\rvert \cdot \lvert\Delta b\rvert + \lvert\Delta a\rvert \cdot \lvert\Delta b\rvert$, shifted, summed

A step moves a wave's phase by up to d = min(2,048 · n, 2^{31}) units. The sine's steepest slope, π/2 per unit in Q1.30, is below 1,609/1,024, and the table adds at most 4: W(n) = min(((1,609 · d) ≫ 10) + 4, 2^{31}). A decay's x moves by at most dx = ((λ · Δτ) ≫ r) + 1 a step, with Δτ = min(max(2,048 ≫ s, 1), τ's largest), and 2^−x's slope is below 178/1,024 per unit, plus 3 for rounding: E = min(((178 · dx) ≫ 10) + 3, 2^{30}) and D = min(|a|, ((|a| · E) ≫ 30) + 1); D = 0 where λ = 0. For a relaxation, τ's largest is taken over the band (to the next band's start or the window's end), and the jump at a switch is |the next band's from − this band's value at the switch|.

4.6 Certificates

A cached fact F reads laws L1 … Ln and names, for each, a band $[lo_i, hi_i]$ of values for which F stays true (a threshold is a band open on one side; "until the law ends" is one too). F's certificate is c(F) = min over i of first_outside(L_i, $[lo_i, hi_i]$, from now); its validity window is [now, c(F) − 1]. Steps are read at their start; the world clock is read through the world's rate (Figure 3).

Reads between steps (for caches read at drawing time). Drawing reads laws at a step plus a fraction. Such a cache searches with each band shrunk by the law's largest rate, [lo + r, hi − r]: a value inside the shrunk band at a step's start stays inside [lo, hi] at every fraction of that step, since two times at most a step apart differ by at most r. Ending the window a step earlier would not do: a value near an edge could leave and return within one step. The driver grid needs no shrinking: its sweeps take n · r over a whole horizon of n steps, which covers every fraction.

```
first_outside(L, lo, hi, from, until, limit):
  if L.envelope ⊆ [lo, hi]: return NEVER                    // no read at all
  s ← from; reads ← 0
  loop
    if reads = limit: return NOT_BEFORE(s)                    // queued as a re-check
    v ← value(L, s); reads ← reads + 1
    if v < lo or v > hi: return AT(s)
    if L has ended at s or L.rate = 0: return NEVER          // the value holds from
here
    if L.kind = RELAXATION:                                   // by inversion
        e ← min(the last step of s's band, until)
        if value(L, e) is outside: return AT(first outside step in (s, e], by
halving)
        s ← e + 1
    else:                                                      // by the largest rate
        s ← s + ⌈min(v − lo + 1, hi − v + 1) / L.rate⌉
    s ← max(s, the first step after L starts)                 // before t0 the value
holds
    if s > until or s > L's end step: return NEVER
```

Why it is never late. In j steps a law moves at most j · r, so from a value g away from the band's edge it cannot leave before ⌈g/r⌉ steps: the search steps through time by a rate bound as sphere tracing steps along a ray by a distance bound [25]. A relaxation band moves one way only, so if it is inside at both ends of a band it was inside throughout. On a world clock running at p/q, the jump is to the first step whose world time has moved more than ⌈g/r⌉ − 1 steps. The read limit (64 by default) bounds a queue's work in a step; it changes when the work is done, never which step comes out. The same search finds when a threshold is first reached; when a law ends is found by halving on its clock.

4.7 Two queues

Both queues are one structure: an ordered multiset of entries with counts (Figure 4). An entry is (step: i64; kind: u8, 0 a law ended, 1 a threshold reached, 2 a band left, and further kinds such as contact, footstep and intent; id: an ordered key; what: the law's first word (u32) and a threshold (direction, i32)

or a band (lo, hi: i32); sure: 1 an event, 0 a re-check, meaning search again from this step) with a count (u64). Entries sort by (step, kind, id, what, sure), a re-check before an event with the same rest. A second index maps (kind, id) to its entries. Nothing in a queue is saved or sent.

Queue	Where	Holds	Cap a step
Gameplay	One per area, on the CPU, the same on every machine; ids are stable world identities	Events that decide results: a law ends, ice reaches its melting point, a door finishes opening	N (≥ 1, per area); the rest carry over to the next step, in order
Cache	One per machine; ids are any key (a driver's number)	Events that change only cost: a window ends, a listing must be renewed	None

Caches differ between machines (settings, hardware, what is in view), so their events must never touch a result. The gameplay queue must hand out the same events in the same order on every machine, as deterministic lockstep requires [28], whatever order they were added in and whatever the thread count and read limit.

Operations: **push** an event at its step; **arm** a search from a step; **pop** the next event up to a step; **withdraw** every entry of a (kind, id), through the index.

```
pop(to):
  loop
    if now > to: return NONE
    if the cap is reached at now: now ← now + 1; taken ← 0; continue     // carry
over
    e ← the first entry; if none: now ← to + 1; return NONE
    if e.step > now: now ← min(e.step, to + 1); taken ← 0; continue
    if e.sure: take one of e's count; taken ← taken + 1; return (now, e)
    remove e; r ← the search for e.what from e.step (§4.6)              // a re-check
    if r = AT(s): insert (s, e.kind, e.id, e.what, event) with e's count
    if r = NOT_BEFORE(s): insert (s, …, re-check) with e's count
```

By construction: nothing is refused or lost (a multiset); what a re-check finds sorts after it, so the order holds; carry-over keeps the order; taking events up to step N at once equals taking them a step at a time; withdrawing a (kind, id) gives exactly what would have come out had those entries never been queued, carry-over included.

4.8 Folded areas

An area with nobody in it goes awake → asleep → settling (up to about 30 s) → folded, keeping a snapshot, its law table and its gameplay queue, at no cost: its laws are closed forms of time. When someone returns at step R, every law is evaluated at R and the queued events up to R run in queue order, each outcome a closed form at its own step (the ice melted at step N), proposed to the area's authority like any other cause. Passing state, such as a cooling stone, is folded by its closed form when the area streams out. Only physical causes fold and wake an area, never the camera.

4.9 Caches with validity windows

A cache is anything worked out from matter and kept. Each entry carries a key (what it was made from) and a validity window (for light caches, 8 bytes: from and to as 32-bit step counts from a base step). The window ends at the entry's certificate, which waits in the cache queue; when it comes, the entry is checked or rebuilt. Caches are never saved, hashed or sent, so no cache can make two machines' results differ. Caches of yes-or-no facts (lists, verdicts, certificates) change the cost, never the picture: inside its window such an entry is exactly true, the path without the cache stays as the reference, and a verify mode counts disagreements, with 0 the bar. Continuous caches (sky tables, probe light) may differ from a fresh computation by at most a written budget of one eighth of an output step; their window is the time the bounded change stays inside that budget, and all of a pixel's bounded approximations together stay below one output step. For the light caches: light parts kept apart from geometry parts, so a turning sun relights without re-tracing; and still caches that never switch off for motion, because they cover still matter only and moving things enter through their envelopes (DP1's mover bins for light; the driver grid

for rays, §4.14).

Cache	Its band
A law frame's listing in the driver grid	Its sweep over a horizon
A moving frame in the particle solver's list	Its sweep can reach the solver's box
Records decoded into their frame's space	The record unchanged, its object still moving
Sky tables; probe light	The sun's direction law: rebuilt before the bounded change could pass its budget
A shadow bound per sun cone; sun verdicts with swaying casters (outer boxes grown and inner boxes shrunk by the caster's envelope; DP1)	The sun inside its cone; the caster inside its envelope

4.10 Drivers and frames

Frames move; physels rest. Frames sit inside frames by a parent link of any depth (for example region → object → bone or branch). A physel stores its position, rotation and size in its innermost frame, so its record never changes while its thing moves; only the frame's pose does (Figure 5). Drivers: **Still** (never moves); **Rigid** (a rigid solver, Live while awake, Still asleep); **Law** (recipes over laws, §4.11); **Articulated** (bones from intent, gait phase and a root path, posed as dual quaternions [27]); **Modal** (an archetype's modes forced by a wind law, §4.15); **Solver** (a solver's island, published as law segments).

Record	Fields
Frame	Position (3 × f32), rotation (4 × f32), scale, velocity, spin, previous pose, driver (u32; all ones: none), parent (u32; all ones: the world)
Driver	Kind, time form, start time (the time its laws count from), parts (parents before children), alive; 64 bytes in all
Part (a law frame)	Frame (u32), parent (u32), recipe (12 × u32), rest position (3 × f32), rest rotation (4 × f32)

A law driver is refused when a part's parent is not the world, an earlier part, or a frame no law moves, or when a frame it would move holds another driver's law frame. So every chain starts at a holder no law moves, and all chains can be posed in one pass with no order between drivers.

4.11 Local poses in integers, then f32 in one order

Recipe (12 words)	Words	Local pose
Fixed	0	The rest pose
Hinge	1; pivot (3 × i32, 2^{-24} m, from the rest origin in the parent's axes); axis (3 × i32, unit, Q1.30); angle law (exponent −32: 2^{-32} turns)	$h = \theta \gg 1$; q = unit(axis · sin h, cos h); offset = pivot − R(q) · pivot
Loop	2; n (whole turns per 2^{21} steps); phase; yaw0; radius (2^{-24} m); bob law (exponent −24) or none; body origin (3); circle centre (3)	a = n · t32 + phase; q = the yaw (yaw0 − a) about up; offset = centre + R(q) · body + (r cos a, bob, r sin a)

Lengths are at most 2^{28} units (16 m) a part, so offsets stay below 2^{30}. A loop's centre is split between the frame's rest position (f32) and an integer remainder, so a loop 30 m out keeps its centre to 2^{-24} m, not f32's 2 μm. Q1.30 products are 64-bit and shifted once. A rotation made from table sines is made unit by one integer Newton step, $g = (3 \cdot 2^{60} - |q|^2) \gg 31$ and $q \leftarrow (q \cdot g) \gg 30$, leaving it unit within about 10^{-9} (unchecked, the tables could shrink it by 3×10^{-7}).

The integer pose becomes f32 exactly: each part is rounded in integers to 24 significant bits (halves away from zero), converted, and scaled by a power of two. Poses then compose in f32 in one fixed order: local = (rest position + offset, q ⊗ rest rotation); world = (parent position + rotate(parent rotation, local position), parent rotation ⊗ local rotation), a vector turning as $t = 2(u \times v)$, $v' = v + w \cdot t + u \times t$. No fused multiply-add, square root or normalising; any part below 2^{-126} is written as +0, since GPUs may flush it. On the GPU each f32 multiply, add and subtract of the composition and of the decode site (§4.13) carries SPIR-V's NoContraction decoration, written into the shader directly because a shader compiler may drop the source language's "precise" marker; a test checks that none is fused. The CPU uses no fused multiply-

add. So the CPU's and the GPU's poses have the same bits. The same integers composed in f64 give the exact pose; velocities are central differences of exact poses 512 ticks (about 0.52 ms) either side, plus the holder's motion.

4.12 The pose pass

Once a frame, at the predicted display time (in VR, the headset's), once for both eyes, before any reader. The clock arrives as integers (step in two words, fraction) for both clocks. Each law frame has an entry of four 16-byte planes and a 16-byte tail; plane j of entry e sits at j × count + e, so neighbouring threads read neighbouring words.

Plane	Holds
0	Recipe words 0-3; word 0 holds the frame's slot above the kind (kind | slot ≪ 2)
1	Recipe words 4-7
2	Rest position (3 × f32); the parent: all ones the world, bit 31 set a frame no law moves (its drawn pose read from the object buffer), else the entry it is inside
3	Rest rotation (4 × f32)
Tail	Recipe words 8-11 (loops only)

```
pose_thread(e):                                        // one thread per law frame
  h ← follow parent links from e to the world or a frame no law moves
  (p, q) ← h is the world ? (0, identity) : the drawn pose of h
  for each law frame x on the chain, root first, ending at e:
      (p, q) ← compose_f32((p, q), integer_local_pose(x, clocks), rest(x))   // §4.11
  write (p, q) to slot first + ⌊frame(e)/2⌋, half (frame(e) mod 2)
```

No thread waits for another, and no level waits for the one above. Poses go into a tail of the instance buffer that starts at slot `first`, after the records' and stand-ins' slots: two 32-byte poses (position, rotation) per 64-byte slot.

4.13 Frame-space records and one decode site

A record is decoded once into a 64-byte instance; a moving object's records are decoded once into its frame's space, and again only when they change or it stops moving.

Bytes	Field
0-15	Centre (3 × f32, in the frame's space when the in-frame flag is set); the shape's dials as one word (sides and ends 4 bits each, taper 8, bevel 8)
16-31	Rotation (4 × f32), the same way
32-47	Half-sizes (3 × f32); material (16 bits) and tint (16 bits)
48-63	The frame (u32); seed, flags, temperature and wetness (8 bits each); second material (16 bits) and blend (8 bits); carve count (bits 0-2) and the in-frame flag (bit 3)

Every reader of a record's place calls one function, the decode site: if the flag is set it reads the frame's pose (p, q) from the tail and returns p + rotate(q, x) and q ⊗ r, each operation marked NoContraction (§4.11). Culling, visibility, the exact cast, shading, sun rays, shadow receivers, the shadow bound, the distance field, heat pairing and the probes all call it, and a source scan in the tests fails on any other read; so every kernel places a moving record with the same bits. An object that has just woken stays on a short world-space list until the grids are rebuilt without it.

4.14 Sweeps and the driver grid

Sweeps. A horizon is n steps from h0, or "for good". A law's span over it is [max(low, v(h0) − n · r), min(high, v(h0) + n · r)], or its envelope for good. A hinge's sweep: the angle range [a0, a1] from its law's span, widened by 2 units for the half angle's lost bit; the box's corners turned about the pivot by the middle angle, grown by the chord 2ρ · sin(half/2) (ρ the corners' largest distance from the pivot, half = (a1 − a0)/2; 2ρ from a half turn on), kept within ±ρ of the pivot. A loop's: the middle angle and half the travel over the horizon; the body's corners turned by the middle yaw and grown by the chord at their radius; the circle's point grown by the chord at radius r; plus the bob law's span. A chain is swept level by level up to the holder, placed by its drawn pose, and padded for f32 by 2^{-16} × (the largest coordinate + 1

m). Each step holds every reachable point: a turn within ±half of the middle moves a point by at most the chord, and offsets add.

Trees. Each moving frame has a binary tree over its chunks' boxes in its own space, each padded for f32, split at the median of the longest side of the chunks' centres (ties by chunk number, so the same tree every time), built when its chunks are made and never refit. A node is 32 bytes: lo (3 × f32); a word naming the first of two adjacent children, or bit 31 and a leaf's chunk; hi (3 × f32); a leaf's member count.

Two layers (Figure 6), each a uniform grid with its own corner and cell size:

Layer	Lists	Rebuilt
Lasting	Law frames held by the world or a resting frame and listed for good: their sweep for good grows their box by at most G each way (G = 2 m, or the box's largest side if more), or no horizon would halve it	Only when its listings change
Current	Live frames at their drawn pose; law frames held by an awake frame, at their pose now; law frames listed for a horizon, by their sweep	Each frame anything in it moved or was re-listed

Horizons and re-list events. A driver's listed parts share the longest horizon of 120 steps (a quarter second) × 2^k, k ≤ 14 (about 68 minutes), over which every sweep grows its box by at most G. A horizon is kept only where it at least halves some part's box for good; otherwise the parts are listed for good (a fast sway covers its envelope within a quarter second anyway). A horizon puts one re-list event, with the driver as its id, in the cache queue at its last step. A change of the laws, of a part's tree or of its holder withdraws the event and lists the driver again; a 64-bit hash of what a listing depends on (the laws' version, the driver's start, each part's frame, parent, recipe, rest pose, listed or not, tree box, holder and holder's pose) shows which drivers changed.

```
list(driver, s):
  parts ← its frames in frame space, held by the world or a resting frame
  own ← each part's box at s;  good ← each part's sweep for good
  fits(sw) ← every part grows by at most max(G, its own largest side)
  if fits(good): return FOR_GOOD
  best ← none
  for k ← 0 … 14:
      sw ← the sweeps over [s, s + 120·2^k)
      if not fits(sw) and best ≠ none: break
      best ← (120·2^k, sw);  if not fits(sw): break
  if no part's best box is at most half its box for good: return FOR_GOOD
  queue re-list(driver) at s + best.steps − 1;  return best
```

Cells. A layer's cells (x fastest) are (first item, count) pairs of u32; items are (frame, root node) pairs of u32, each cell's in frame order, filled by prefix sums in frame order, so one thread and many give the same grid. A box is listed in every cell its padded box touches plus 10^{-3} of a cell beyond, so the cell a shader computes in f32 for any point inside lists it. The cell size starts at the entries' median largest side (at least 0.25 m) and grows by a quarter until there are at most four cells an entry plus 64, and at most 1,023 along each axis. Each layer's corner, cell size and packed size (10 bits an axis) ride in the frame's constants. The CPU builds the grid; each frame the GPU only receives a copy of the current layer.

The walk. Secondary rays in shading and in the light probes walk moving frames in a loop of their own, before the still world's walk, which is unchanged and starts from the best distance found. The walk steps each layer's cells along the ray. A frame is walked once per ray, in the first cell along the ray that lists it: its cells are one run along the ray (its box is convex), so a frame the previous cell listed is skipped. Inside a frame the ray is turned into the frame's space for the tree's boxes and the records' bounding spheres, nearer child first. Boxes and spheres grow by a margin for what turning the ray in f32 may stray: 2×10^{-6} × (the largest coordinate of the ray's origin less the frame's position, plus the largest coordinate of the frame's position) + 10^{-6} m; a record's sphere has its half-size vector's length as its radius, plus that margin. Each record that passes is cast in world space through the decode site. Shading kernels holding this walk run only in frames where some record is in its frame's space; other frames use kernels without it. The particle solver also uses sweeps: it lists a moving frame only while the frame can reach the solver's box.

4.15 The modal-instance law

Trees of one species sway alike. An archetype is a damped-modes law in the table; an instance (kind 7) has no header and takes 5 + m words: word 0, the kind, t0's fraction (bits 3-13) and a frequency factor k in 2^{-16} (bits 14-31: 1 to $2^{18} - 1$); words 1-2, t0's step; word 3, the archetype's first word; word 4, c; then per mode its share of the archetype's amplitude (i16, 2^{-15}) and its phase (u16, 2^{-16} turn). Its value is, to the bit, the damped law it stands for: $a = (A \times \text{share}) \gg 15$, $\varphi = \text{phase} \times 2^{16}$, $n' = (n \times k) \gg 16$, $\lambda' = (\lambda \times k) \gg 16$, with the archetype's clock, exponent, end, s and r and the instance's own t0 and c. A copy at another size sways at another rate ($k \propto 1/\text{size}$) and decays in step. Its bounds are the expanded law's, worked out on the CPU. A four-mode instance is 9 words, against 26.

4.16 Each frame, in order

1. The clock gives the time the frame is drawn for, as integers.
2. CPU: law frames posed for CPU readers, in parallel with results in driver order, written into their objects without marking them moved (nothing is re-sent or re-stamped for them).
3. CPU: the driver grid: trees for frames new to frame space; the cache queue drained to now; drivers whose hash changed or whose event came listed again; layers rebuilt as needed; changes uploaded.
4. GPU: changed records decoded into frame space, and the pose pass; then culling, visibility, shading and probes, each read through the decode site, with the driver walk in secondary rays.

Laws are made, bounded and searched, and both queues kept, on the CPU only; laws are evaluated on the CPU for gameplay and caches, and on the GPU wherever a shader reads one.

4.17 Parameters and useful ranges

Parameter	Value	Useful range
Step; fraction	1/480 s; 11 bits	1/120-1/1,920 s; 8-16 bits
Terms in a law; search read limit	64; 64	8-256; 1-1,024
Gameplay events a step	Per area	1-10^4
Shortest horizon; doublings	120 steps; 14	30-480 steps; 8-20
Growth limit G	2 m, or the box's largest side	0.25-8 m, or a share of the box
Smallest cell; cells an entry	0.25 m; 4 (+ 64)	0.05-2 m; 1-16
f32 pad	2^{-16} × (largest coordinate + 1 m)	2^{-20}-2^{-12}
Velocity half-window	512 ticks	64-4,096 ticks
Recipe lengths	2^{-24} m, ≤ 16 m a part	2^{-30}-2^{-16} m
Frequency factor; share; phase	2^{-16}; 2^{-15}; 2^{-16} turn	8-24 bits each

4.18 Error bounds and how each is proven

Bound	Proof
A law stays in its envelope and moves at most its rate a step	The bounds follow the evaluator's integer operations, rounding included
A certificate is never late: at step starts, and at every fraction with each band shrunk by one rate	The rate argument; one-way bands for inversion
Queue order does not depend on insertion order, read limit or draining	A total order on a multiset; re-checks sort before events
CPU and GPU give the same law values and poses	Integer twins of the same words; exact f32 conversion; one order; flushing
Sweeps hold every reachable pose	Chord and rate bounds; f32 padding
A rotation after the Newton step is unit within about 10^{-9}	One Newton step squares the error

5. Variants

The method may also:

- **Use other laws:** splines and B-splines, piecewise polynomials of any degree, Chebyshev or

Fourier series of any period, wavelets, noise functions with stated Lipschitz bounds, ephemerides, timetables, state machines with timed transitions, seeded random processes with bounded sample paths, solutions of linear differential equations (forced oscillators, relaxation, drag), learned functions with certified Lipschitz bounds, and sums, products and compositions of these; stored as bytes, 16-, 64- or 128-bit words, floating point under fixed rounding, rationals or intervals, or bytecode; with parameters shared per species, material, mesh or clip.

- **Bound and certify otherwise:** interval or affine arithmetic, Taylor models, second-derivative bounds, piecewise envelopes, tables of per-segment extremes; crossings by exact roots, Sturm sequences, bounded Newton steps or bisection; one certificate over several laws jointly (the distance between two moving frames); certificates shortened or lengthened when laws change, worked out lazily or eagerly.
- **Queue otherwise:** calendar queues, timing wheels, radix heaps, per-step buckets, sorted GPU arrays, per-thread queues merged in the total order; caps per kind or id; any total tie-break that is the same on every machine; gameplay events, laws and withdrawals sent over a network (DP3) or saved with the world; timers, scripts, growth and decay as events.
- **Certify any cache:** shadow maps and virtual shadow-map pages; visibility verdicts; ambient occlusion; irradiance, reflection and radiance probes and caches; light lists and clusters; distance fields and voxel structures; potentially visible sets; level-of-detail choices and impostors; texture and geometry residency; sky and atmosphere tables; acoustic paths, impulse responses and reverb; broad-phase pairs, contact caches and sleeping islands; navigation meshes with moving obstacles; sight lines; network interest sets (which clients hear about which objects); streaming priorities; offline-render caches across frames. Rebuild, refit or extend an entry when its certificate comes; hold windows as steps, offsets or frame counts.
- **Frame and index otherwise:** any depth of frames; region frames with 64-bit offsets; matrices or dual quaternions; f64 or fixed-point poses; skeletons, inverse kinematics, motion matching, ragdolls, vehicles, cloth anchors and crowds as drivers; any primitive stored in its innermost frame (triangles, meshes, voxels, distance bricks, point clouds, Gaussian splats, particles, curves); moving frames indexed by bounding volume hierarchies, k-d trees, octrees, hashed grids or sweep-and-prune lists, in one or more layers by how long a listing lasts; a hardware ray-tracing top level built from swept instance boxes and rebuilt only when a horizon ends; horizons on other ladders (1/60 s × 3^k) or by screen-space error; poses made on the CPU, on integrated graphics, or in the shader that reads them.
- **Run on other precision and hardware:** any integer width; floating point under strict rounding; any GPU through Vulkan, SPIR-V, Metal, Direct3D 12 or WebGPU; CPUs with or without SIMD; consoles, phones, standalone headsets, FPGAs.
- **Serve uses outside games:** film and animation (frame caches, motion-blur bounds over a shutter interval, crowds and vegetation on laws); visual effects and engineering simulation (expensive solves kept while bounded motion stays in a band); robotics (collision checks against machines and conveyors on known laws, the next check time from a certificate, queues for safety monitoring); digital twins of factories, cities and ports (machines, traffic lights, ferries and a year of shadows as laws, empty areas worked out on demand); CAD (interference checks for mechanisms by sweeps); science and medicine (orbits and visibility windows, tides, breathing and heartbeat as periodic laws with envelopes for imaging or treatment planning); user interfaces (animations as laws, the compositor told when a layer can next change).

6. Worked example

A branch sways on a hinge at 0.26 Hz, amplitude 1.406°. Its angle law: one wave, $a = 2^{24}$ units of 2^{-32} turn, n = 1,136 cycles per 2^{21} steps. W(1,136) = 3,655,652, so the rate is $1 + ((2^{24} \times 3{,}655{,}652) \gg 30) = 57{,}120$ units a step (0.00479°, the true peak speed of 2.297° a second). A cache made at angle 0 holds within ±0.25° (±2,982,616 units). From v = 0, g = 2,982,617 and $\lceil g / 57{,}120 \rceil = 53$: the next read is 53 steps (110 ms) later, where the angle is 0.2523°, outside. The certificate is 53 steps on, found with two reads instead of 54. Near the top of the swing the bound is loose, and searches read more often.

The branch's sweep for good: within ±1.406°, a box 3.2 m from the pivot moves at most the chord 2 × 3.2

m × sin 0.703° = 0.079 m, under G, so it is listed once in the lasting layer. A walker going round a circle of radius 5 m at 1.4 m/s moves about 1.4 m a second: its horizon is 480 steps (1 s), under half its box for good, so one cache-queue event a second re-lists it.

7. Figures

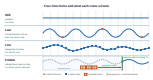


Figure 1. The four time forms and what each costs a frame.



Figure 2. A law's words: the 8-word header and parameters; a modal instance reading its archetype.

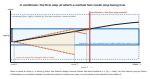


Figure 3. A law's envelope, a cached fact's band and its certificate, found by stepping by the largest rate.



Figure 4. The gameplay queue per area (fixed order, cap, carry-over) and the cache queue per machine (re-checks, withdrawal).

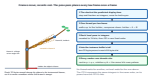


Figure 5. Frames inside frames, records resting in their innermost frame, the pose pass and the one decode site.

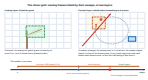


Figure 6. The driver grid: a branch listed for good, a walker listed for a horizon and re-listed by one event, an awake object at its drawn pose, and a ray walking the cells.

8. References

1. J. Basch, L. J. Guibas, J. Hershberger. Data structures for mobile data. ACM-SIAM Symposium on Discrete Algorithms, 1997; Journal of Algorithms 31(1), 1999.
2. L. J. Guibas. Kinetic data structures: a state of the art report. Workshop on the Algorithmic Foundations of Robotics, 1998, 191-209.
3. A. S. Glassner. Spacetime ray tracing for animation. IEEE Computer Graphics and Applications 8(2), 1988, 60-70.
4. J. Chapman, T. W. Calvert, J. Dill. Exploiting temporal coherence in ray tracing, Graphics Interface 1990, 196-204; Spatio-temporal coherence in ray tracing, Graphics Interface 1991, 101-108.
5. J. Lext, T. Akenine-Möller. Towards rapid reconstruction for animated ray tracing. Eurographics 2001 short presentations.
6. I. Wald, C. Benthin, P. Slusallek. Distributed interactive ray tracing of dynamic scenes. IEEE Symposium on Parallel and Large-Data Visualization and Graphics, 2003.
7. I. Wald, W. R. Mark, J. Günther, S. Boulos, T. Ize, W. Hunt, S. G. Parker, P. Shirley. State of the art in ray tracing animated scenes. Computer Graphics Forum 28(6), 2009, 1691-1722.
8. The Khronos Group. The Vulkan specification: acceleration structures (VK_KHR_acceleration_structure), 2020 and later.
9. Autodesk. 3ds Max SDK documentation: "Validity Intervals" and "Cache Validity Intervals"

(online, 2016 and later).
10. P. S. Strauss, R. Carey. An object-oriented 3D graphics toolkit. Computer Graphics 26(2), SIGGRAPH 1992.
11. J. Rohlf, J. Helman. IRIS Performer: a high performance multiprocessing toolkit for real-time 3D graphics. SIGGRAPH 1994.
12. R. Nystrom. Game Programming Patterns, "Dirty Flag". Genever Benning, 2014.
13. Epic Games. Unreal Engine documentation: virtual shadow maps; shadow cache invalidation behaviour.
14. T. Sousa. Vegetation procedural animation and shading in Crysis. GPU Gems 3, chapter 16, Addison-Wesley, 2007.
15. R. Zioma. GPU-generated procedural wind animations for trees. GPU Gems 3, chapter 6, Addison-Wesley, 2007.
16. J. Stam. Stochastic dynamics: simulating the effects of turbulence on flexible structures. Computer Graphics Forum 16(3), 1997, 159-164.
17. S. Chenney, D. Forsyth. View-dependent culling of dynamic systems in virtual environments. ACM Symposium on Interactive 3D Graphics, 1997.
18. B. Mirtich. Impulse-based dynamic simulation of rigid body systems. PhD thesis, University of California, Berkeley, 1996.
19. B. Mirtich. Timewarp rigid body simulation. SIGGRAPH 2000, 193-200.
20. D. R. Jefferson. Virtual time. ACM Transactions on Programming Languages and Systems 7(3), 1985, 404-425.
21. S. Redon, Y. J. Kim, M. C. Lin, D. Manocha. Fast continuous collision detection for articulated models. ACM Symposium on Solid Modeling and Applications, 2004.
22. X. Zhang, S. Redon, M. Lee, Y. J. Kim. Continuous collision detection for articulated models using Taylor models and temporal culling. ACM Transactions on Graphics 26(3), 2007.
23. G. Zachmann, R. Weller. Kinetic bounding volume hierarchies for deformable objects. ACM Virtual Reality Continuum and its Applications, 2006.
24. D. S. Coming, O. G. Staadt. Kinetic sweep and prune for multi-body continuous motion. Computers & Graphics 30, 2006, 439-449.
25. J. C. Hart. Sphere tracing: a geometric method for the antialiased ray tracing of implicit surfaces. The Visual Computer 12(10), 1996, 527-545.
26. J. Tessendorf. Simulating ocean water. SIGGRAPH course notes, 2001.
27. L. Kavan, S. Collins, J. Žára, C. O'Sullivan. Geometric skinning with approximate dual quaternion blending. ACM Transactions on Graphics 27(4), 2008.
28. P. Bettner, M. Terrano. 1500 archers on a 28.8: network programming in Age of Empires and beyond. Game Developers Conference, 2001.

© 2026 Kyle Kimberlin. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/